

nordpool-price-forecasting

Day-ahead electricity price forecasting for the NordPool SE3 bidding zone. Two probabilistic models, a ClickHouse-backed bitemporal data store, and a Streamlit dashboard.

What it does

The pipeline fetches hourly day-ahead prices from ENTSO-E and weather data from Open-Meteo, trains two models, and stores quantile forecasts (q05/q50/q95) in ClickHouse. The dashboard reads those forecasts against actual prices over any selected period.

LightGBM fits three separate quantile regressors on 7 price lags, rolling statistics, calendar features, and weather interactions. Prediction intervals are post-hoc widened using split conformal prediction to achieve 90% marginal coverage regardless of model miscalibration.

LEAR (Lasso Estimated AutoRegressive) fits one LassoCV model per hour - 24 independent models. Hour-by-hour fitting handles the fact that peak-hour and off-peak-hour prices behave very differently; a single model has to compromise between them.

Holdout results - 2026-02-14 to 2026-05-15 (1,787 hours)

Model	MAE EUR/MWh	RMSE	q05-q95 Coverage	Spike MAE
LightGBM	21.61	28.26	79.5% -> 90.1% (conformal calibration)	42.30
LEAR	21.13	27.56	92.3%	36.64

Spike MAE covers hours where the actual price exceeded 100 EUR/MWh (274 hours in the holdout).

Stack

- **Prices:** ENTSO-E Transparency Platform
- **Weather:** Open-Meteo (temperature, wind speed, irradiance)
- **Storage:** ClickHouse via TimeDB SDK
- **Models:** LightGBM, scikit-learn LassoCV
- **Dashboard:** Streamlit + Plotly
- **Deployment:** Docker + Docker Compose

Project layout

```
nordpool-price-forecasting/
|-- dashboard/          # Streamlit app
|-- db/                 # ClickHouse schema and series ID registry
|-- ml/
|   |-- models/         # lgbm.py, lear.py
|   |-- evaluate.py     # Walk-forward evaluation engine
|   |-- run_eval.py     # Evaluation CLI
|   +-- train.py        # Training pipeline
|-- model/              # Trained artefacts (git-ignored)
|-- pipeline/           # Data fetch and feature engineering
|-- Dockerfile
|-- docker-compose.yml
+-- requirements.txt
```

Quickstart (Docker)

```
# 1. Add credentials
cp .env.example .env      # fill in at minimum ENTSOE_API_KEY and TIMEDB_CH_URL

# 2. Start ClickHouse and the dashboard
docker-compose up -d

# 3. First run: fetch historical data and train
docker-compose run --rm app python -m pipeline.fetch_prices
docker-compose run --rm app python -m pipeline.fetch_weather
docker-compose run --rm app python -m ml.train

# 4. Open the dashboard
open http://localhost:8501
```

After the first run, `docker-compose up -d` is all you need. Training skips automatically when the models are less than 7 days old; use `--force` to override.

Local development

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
docker-compose up -d clickhouse
streamlit run dashboard/app.py
```

Training and evaluation

```
# Train (skips if models are fresh)
python -m ml.train
```

```
python -m ml.train --force --note "Added 336h lag"
```

```
# Evaluate on the recorded holdout
```

```
python -m ml.run_eval
```

```
# Walk-forward evaluation (retrains on every fold - takes hours)
```

```
python -m ml.run_eval --walk-forward --train-days 365 --test-days 30 --step-days 90
```

Results append to model/MODEL_LOG.md.

Environment variables

Copy .env.example and fill in:

Variable	Description
ENTSOE_API_KEY	ENTSO-E Transparency Platform API key
TIMEDB_CH_URL	ClickHouse connection string, e.g. <code>http://user:pass@localhost:8123/se3db</code>
SE3_LAT / SE3_LON	Coordinates for weather fetch - Stockholm: <code>59.33 / 18.07</code>
SE3_TRAIN_START	Start of historical data, e.g. <code>2020-01-01</code>
MODEL_DIR	Where model artefacts are saved (default: <code>model/</code>)

Bitemporal storage

Each forecast row has two timestamps: `valid_time` (the hour being predicted) and `knowledge_time` (when the model produced the prediction). Querying both lets you ask “what did the model predict for Tuesday 14:00, given Monday’s training run?” - useful for backtesting without lookahead and for tracking how predictions change as more recent data arrives.

License

MIT